1. Background Simulator (Real-time Telemetry)

How it works

To simulate a live city, the platform uses **APScheduler** (Advanced Python Scheduler) to run a background job inside the Flask application context. Every few seconds, this job generates random, realistic changes to traffic, waste, and environmental data, and writes these new rows into the SQLite database. This gives the frontend real data to visualize over time without needing physical IoT sensors.

How to code it (Python/Flask)

```
from apscheduler.schedulers.background import
BackgroundScheduler
from simulator.city_data import generate_tick

def start_simulator(app):
    def tick():
        with app.app_context():
            generate_tick() # Generates and commits DB rows

    scheduler = BackgroundScheduler()
    # Runs the tick function every 6 seconds
    scheduler.add_job(tick, "interval", seconds=6)
    scheduler.start()
```

2. Machine Learning: Traffic Forecasting

How it works

The traffic prediction model looks at the history of a specific traffic zone. It extracts the **vehicle_count**, **average_speed**, and **congestion_level** for sequential timestamps to create training features (X) and labels (y). It trains a **RandomForestRegressor** on-the-fly to predict congestion 30 minutes into the future. A Random Forest is used because traffic patterns are highly non-linear.

How to code it (Scikit-Learn)

```
import numpy as np
from sklearn.ensemble import RandomForestRegressor
```

```
# X = [[count, speed, current_congestion], ...]
# y = [future_congestion, ...]
model = RandomForestRegressor(n_estimators=80, max_depth=6)
model.fit(X, y)

# Predict based on the absolute latest reading
features = np.array([[latest_count, latest_speed,
latest_congestion]])
predicted_congestion = model.predict(features)[0]
```

3. Interactive Leaflet Map Integration

How it works

The dashboard renders an interactive map of Bengaluru using the open-source **Leaflet.js** library and OpenStreetMap tiles. Traffic data is fetched via REST API, and for every traffic zone, a colored circle marker is drawn. Green means low congestion, yellow is moderate, and red is heavy.

How to code it (Vanilla JavaScript)

```
// 1. Initialize the Map
var map = L.map("trafficMap").setView([12.9716, 77.5946],
11);
L.tileLayer("https://{s}.tile.openstreetmap.org/{z}/{x}/{y}
.png").addTo(map);

// 2. Add Markers based on API data
fetch("/api/traffic").then(res => res.json()).then(data =>
{
    data.forEach(reading => {
        var color = reading.congestion_level >= 70 ?
"#e15a4f" : "#51b78b";
        var marker = L.circleMarker([reading.latitude,
reading.longitude], {
            radius: 9, color: color, fillColor: color,
fillOpacity: 0.88
        }).addTo(map);

        // Add a tooltip popup
        marker.bindTooltip(reading.name);
    });
});
```

4. Theming: Dynamic Dark Mode

How it works

Dark mode isn't implemented by writing two separate CSS files. Instead, it uses **CSS Custom Properties (Variables)**. Colors like background and text are mapped to variables (e.g., --bg, --text). When a user clicks the toggle button, a `data-theme="dark"` attribute is added to the HTML root, which reassigns those variables to darker hex codes.

How to code it (CSS & JS)

```
/* Default (Light Mode) */
:root {
  --bg: #f3f4f6;
  --text: #111827;
}

/* Dark Mode Overrides */
[data-theme="dark"] {
  --bg: #111827;
  --text: #f9fafb;
}

/* Applying it */
body { background: var(--bg); color: var(--text); }

// JavaScript Toggle Logic
button.addEventListener("click", function() {
    if (document.documentElement.getAttribute("data-theme")
=== "dark") {

document.documentElement.removeAttribute("data-theme");
        localStorage.setItem("theme", "light");
    } else {
        document.documentElement.setAttribute("data-theme",
"dark");
        localStorage.setItem("theme", "dark");
    }
});
```